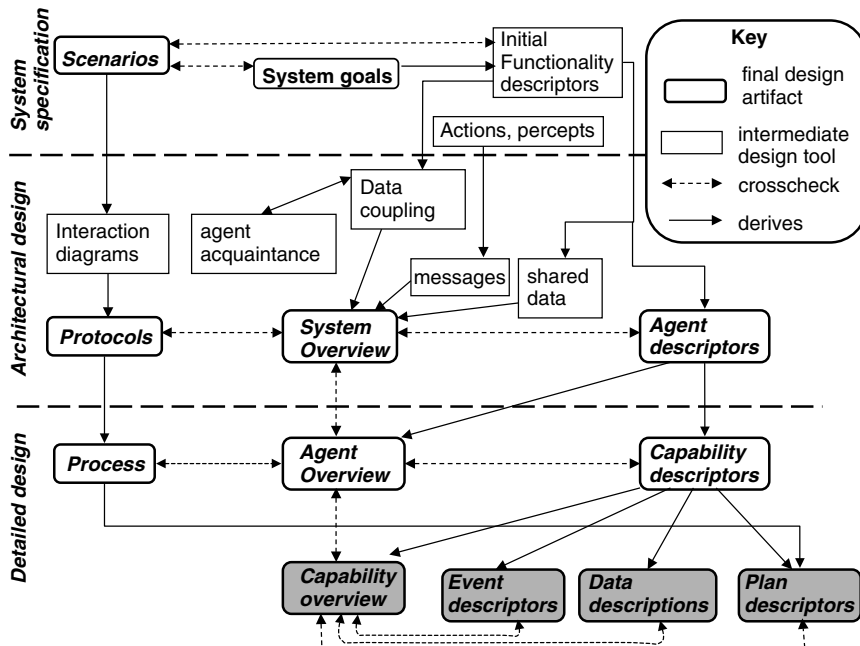


Detailed Design: Capabilities, Plans and Events



Phases, artifacts and relationships in the design process

At this final stage of the detailed design, each capability is broken down either into further capabilities or, eventually, into the set of plans that provide the details of how to react to situations, or achieve goals. At this stage, a number of details regarding the implementation platform become important. We focus on Belief-Desire-Intention (BDI)

platforms in particular, which are characterized by a representation that has hierarchical plans with triggers, and a description for each plan that indicates the context in which it is applicable. BDI systems choose among the plans that are applicable, backtracking to try another plan if the one initially chosen does not succeed.

Although a number of the issues we address at this point are specific to the way BDI systems work and are written, the principles are easily adapted to a range of other agent systems. In particular, any system using agents with plans that react to events can easily be accommodated. At the same time, if the precise development platform is known, then additional issues can be considered and noted during design.

The details we provide here are oriented towards BDI systems, but not towards any particular BDI implementation platform. We do, however, provide examples of additional detail that could be added for a particular platform, using JACK Intelligent Agents as an example platform.

The main steps covered in this final stage are as follows:

1. Further decomposition using capability overview diagrams
2. Sub-tasks and alternative plans
 - Identifying context conditions
 - Coverage and overlap
3. Developing the events and messages
4. Action and percept details
5. Details of data.

9.1 CAPABILITY OVERVIEW DIAGRAMS

In the previous chapter, we developed agent overview diagrams. A further level of detail is provided by capability diagrams that take a single capability and describe its internals. At the bottom level, these will contain plans, with *internal messages* providing the connections between plans, just as they do between capabilities. At intermediate levels, they may contain nested capabilities or a mixture of capabilities and plans. These diagrams are similar in style to the system overview and agent overview diagram, although one of the incoming messages to a plan needs to be identified as the triggering message. Each plan must have exactly one triggering message.

Figure 9.1 shows a capability diagram for the *Stock managing* capability of the Stock Manager agent shown in the previous chapter (Figure 8.2 on page 103). This capability was quite large, as can be seen from the number of incoming and outgoing items on the Stock Manager agent overview diagram from the previous chapter. Consequently, we choose to break it down into the three further capabilities, *Ordering*, *Delay handling* and *Handling new stock*.

Figure 9.2 shows the plans that realize the *Ordering* capability. Consistent with the parent diagram of the *Stock managing* capability, it has one incoming message, two outgoing messages, an action and four external data sources. In addition, it introduces one new data source internal to this capability – the *Monthly Order*.

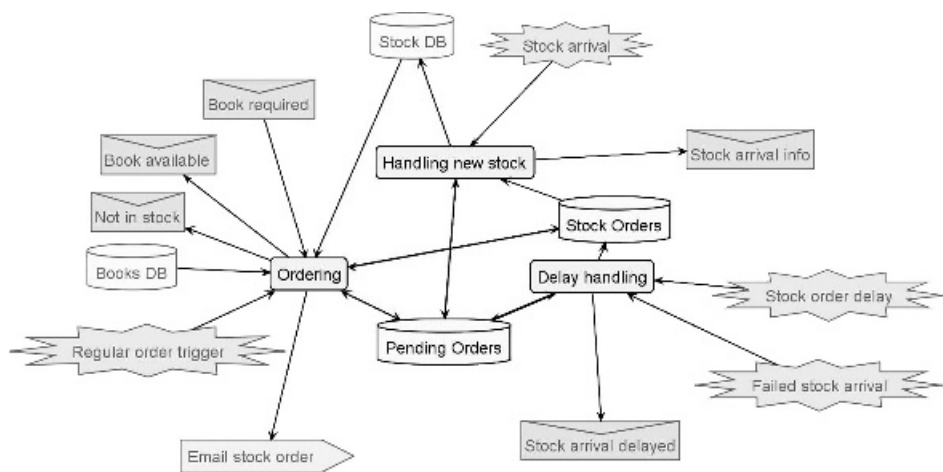


Figure 9.1 Capability diagram: Stock managing capability

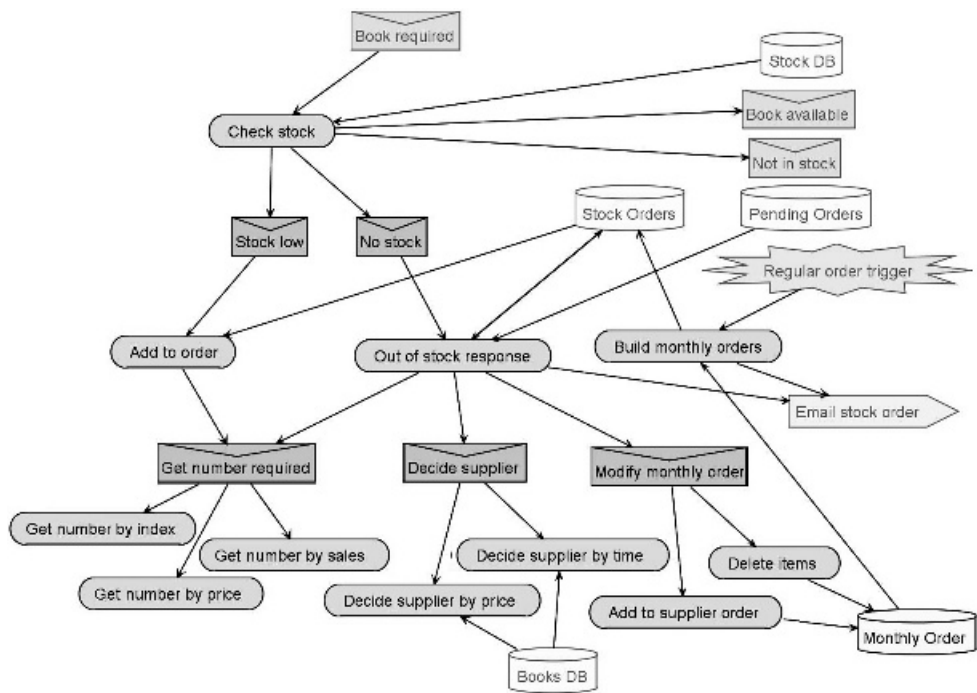


Figure 9.2 Capability overview diagram: Ordering capability

9.2 SUB-TASKS AND ALTERNATIVE PLANS

At the lowest level of detail, each incoming message to the capability must have one or more plans that respond to that message. Multiple plans responding to the message provide multiple ways of reacting to the situation. This allows each individual plan to be simple and straightforward, responding to the relevant message for a particular situation.

Each plan can typically be broken down into some number of sub-tasks, each of which is represented by an internal message. Depending on the implementation platform, sub-tasks can be combined using programming control structures – sequence, parallel, if, or, and so on. These control structures are not shown in the capability overview diagram – like the other overview diagrams, it captures only static structure.

Electronic Bookstore: Case study

In Figure 9.2 of the *ordering* capability, we see that the *Check stock* plan has four messages or sub-tasks: *Stock low*, *No stock*, *Book available* and *Not in stock*. These actually represent two cases requiring ordering of stock, as well as a third case not requiring stock ordering. In one case there is no stock, with an internal message generated to handle this situation and a message sent to the entity requiring the book, stating that it is not in stock. In the second case, stock is low, and so a message is sent back saying the book is available as well as an internal message being generated to handle the low stock situation. The third case (not requiring any ordering of stock) simply sends the *Book available* message.

The two stock-ordering situations trigger two separate plans to deal with the two situations. Figure 9.3 shows the items relevant to the situation in which stock is low. In this situation, the *Stock low* message triggers a plan *Add to order*, which adds the book to the monthly order form, after first checking in the *Stock orders* DB that it is not already ordered and awaiting delivery, and the *Monthly Order* DB to ensure it is not already on this month's order list. *Add to order* then has three sub-tasks: *Get number required*, which determines the number of books to order, *Decide supplier*, which determines which supplier to order from and *Modify monthly order*, which adds the required number of books to the monthly order for the relevant supplier.

The *Get number required* message is an example of a situation in which there are several different plans or ways of achieving this sub-task. *Get number by index* simply obtains the number on the basis of a categorization of the book, and a standard number of copies to order for this book category. *Get number by price* determines the number of copies to order on the basis of the price of the book – perhaps not wishing to have too much money tied up in many copies of expensive books. *Get number by sales* looks at the sales history for the book and determines an order number on the basis of this information. The availability of these different plans, allows the system to be flexible and choose depending on circumstances. For example, if

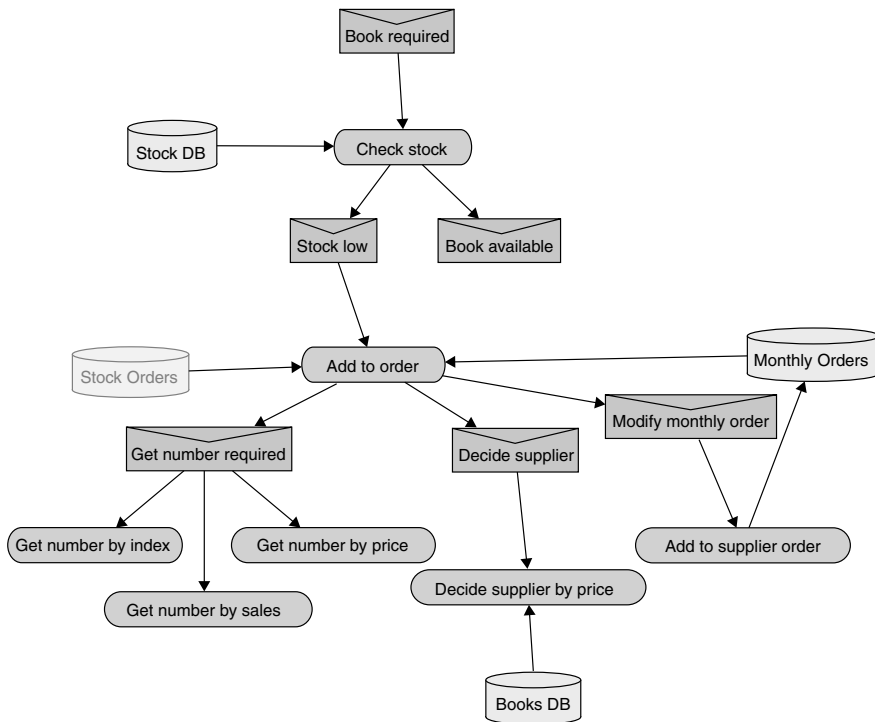


Figure 9.3 Low stock situation within the Ordering capability

there is a cash flow problem, it may be most appropriate to use *Get number by price*. This decision can be made at run time. (We note that each of these plans would actually need access to data that is not currently shown in the diagram).

It is often the case that initially only one plan may be provided for achieving a particular sub-goal. Usually, this is the most straightforward or standard way of doing something. Later, additional plans can be added to address more unusual situations, or to provide alternative approaches that can be useful in some situations.

9.2.1 IDENTIFYING CONTEXT CONDITIONS

Each plan is triggered by a specific *event*. This event may be the arrival of a percept, arrival of a message from another agent, or an internal message or sub-task within the agent. We use the term event to include each of these, as it is also a commonly used term in many agent systems. If there are several plans that could be triggered by a given event, then it is important to specify the conditions or situation under which the various plans are applicable. We call this the *context condition*, and in many agent systems (and all BDI systems), this maps quite directly to an implementation concept.

APPLICABILITY

The context condition most often represents information about the state of the environment, which makes the plan suitable for use in that situation. For example, a plan to recompense a customer for a lost book by immediately sending a new book may have as a context condition that the required book is in stock. An alternative plan may provide the customer with a refund and have as a context condition that the book is not in stock. Additional plans providing more options may be added later.

Information required in context conditions has implications for other parts of the design, in particular, data representations. Exactly what is possible will depend to some extent on the implementation platform. Some systems provide a specialized data structure for representing agent beliefs, which can then be accessed in context conditions. There may also be internal data structures in the host programming language, which contain information about the environment. There may also be functions that can be run as needed as part of a check of context condition. Care should be taken to specify context conditions precisely at this stage and to ensure that the design includes the relevant data.

MULTIPLE BINDINGS

Many BDI systems provide the ability to bind variables as part of a context condition, giving the possibility of multiple plan instances, with different bindings of some variables. For example, a plan to courier a book requires that the destination be in an area that is served by a courier in the database. There may well be several such couriers. Binding of a courier variable to these different options gives multiple plan instances, each using a different courier. This gives flexibility and robustness, in that if a plan to use one courier fails, because perhaps they do not respond, then another plan (with a different courier) will be tried.

FULL SPECIFICATION

In some cases, a context condition should be specified even if there is only one plan available. If there is some situation that is necessary for the successful execution of the plan, then this should be specified as a context condition, even if there is no other option available. (By doing this, the system is able to efficiently and effectively recognize a failure situation, and possibly remedy it by making other choices elsewhere.)

Sometimes, inexperienced agent programmers fail to represent context conditions that they expect to be true, on the basis of previous actions. However, it is important to remember that an agent does not have complete control of the environment, and external events can change the environment during the time an agent is pursuing a series of plans towards a particular goal. Representation of necessary context conditions helps ensure the correct functioning of the agent.

Electronic Bookstore: Case study

Returning to the example of the different plans for *Get number required* in our example in Figure 9.2, we could define context conditions as follows:

Get number by index

Context: Book has category assigned

Data requirements: Category field in Stock DB; Data structure (*Ordering-categories*) mapping categories to a default number to order.

Get number by price

Context: Wholesale book price greater than \$100

Data requirements: Wholesale book price as a sub-field of supplier in Books DB.

Get number by sales

Context: Sales for month greater than default number orders for category

Data requirements: Data structure (Monthly sales) that holds number of sales per book; structures for determining default (index) order values as for *Get number by index* plan.

9.2.2 COVERAGE AND OVERLAP

Having multiple plans to respond to an event is a powerful mechanism that facilitates robustness, flexibility and modularity. However, it does require that the designer consider potential interactions between the plans. *Coverage* is the term we use to refer to the concept of whether, for a given event, there will always be some plan with a matching context condition. *Overlap* refers to whether it is possible, in some situations, to have more than one plan that is applicable, necessitating a choice between them.

If there is a plan with no context condition, then coverage is guaranteed. It can be useful to have such a plan to ensure coverage. However, it is important, as noted previously, to not ignore (and fail to represent) conditions that are actually required for successful execution, but which one expects will hold. Coverage can also be guaranteed if there is a pair of plans whose context conditions are the complement of each other. For example, if one plan has a context condition that book price is less than \$100, and another has a context condition that book price is greater than or equal to \$100, then there is coverage. If, however, the second plan has only book price greater than \$100, then (if these are the only two plans) there is not (complete) coverage, as in the case where book price is exactly \$100, neither plan will be applicable. It is important for the designer to do a careful analysis of coverage to ensure that, unless intended otherwise, there is no gap in coverage. If it is intended that there be no response in some situations, then a gap in coverage may be appropriate. However, it is important to check how this is handled by the implementation platform and to implement accordingly; in some systems, it may be necessary to have an 'empty' plan that does nothing, to obtain the desired behaviour.

A related concept to coverage is that of *overlap*. We say that there is overlap in response to a trigger if it is possible for there to be multiple plans that are applicable in some situation. There are two mechanisms by which this can happen. One is when two context conditions overlap, that is, there is some situation in which both could be true simultaneously. The other is where the context condition contains variables that can potentially take on multiple values at a single point in time, leading to multiple plan

instances of a single plan type. Overlap is a kind of redundancy and can be very useful in building a system that is robust and able to recover from failures. However, it is very important that it be considered and understood at design time.

 **TIP:** Interactions between context conditions that have not been foreseen are one of the very common bugs that arise in agent systems. Sometimes there is a failure or lack of response, due to an unintended gap in coverage. At other times, there is an unintended overlap in context conditions, leading to a different plan being executed than that which was intended. Careful and systematic analysis during detailed design helps eliminate these bugs.

Preferences

If there is overlap, one plan will be chosen (initially) from all those that are applicable. It may be that although several plans are viable, there are preferences as to which should be tried first. The mechanisms available for specifying and controlling this vary from system to system. However, it is important that the designer consider whether such preferences exist, and if so, what they are. If the implementation platform lacks a mechanism for controlling the order in which plans from an applicable set are chosen, then the designer must weigh up the advantages of controlling ordering by tighter specification of context against the advantages of robustness by not excluding less preferred but possible plans.

9.3 EVENTS AND MESSAGES

As indicated, we use the term *event* to denote things that can trigger the choice and execution of a plan. These can be the messages that we have identified between agents, the percepts coming from the environment, or the messages internal to an agent, indicating sub-tasks or messages between plans and/or capabilities.

The most important issue associated with the events is the precise specification of the information carried by that event. Again, the exact way in which this is carried is dependent on the implementation platform. The important design issue is to specify clearly what information is required as part of the event. Both the type of information, and, where relevant, the allowable values should be specified. In some cases (for percepts or messages between agents), this will have been specified at an earlier stage. However, it may need to be refined or added to during detailed design. In addition, it is important at this stage to add additional information such as that regarding coverage and overlap. Many new events (or internal messages) are developed as the plans take shape and sub-tasks are identified.

For example, if there is a book order event, presumably it is important for that event to carry the information as to which book is being ordered. It may also be necessary to carry information about who is ordering and the delivery address. This information has possibly been identified during architectural design. As a plan is being developed, it

becomes evident precisely which information is required, and as a result, the information to be carried by the event is refined. Some events may only serve as a trigger and may not need to carry additional information.

For messages between agents, it is also important to specify whether a reply is expected and the details of that reply.

Events representing percepts from the environment must specify the information carried as part of the percept. This may have been extracted from raw data as part of a processing phase.

Electronic Bookstore: Case study

Referring to Figure 9.3, we see six different events. Many of these events will need to carry the information regarding the book under consideration. The exception is the *Book available* message, which is a response to a request, and therefore may not need to explicitly carry this information.

The *Modify monthly order* event would need, in addition to book id, to carry the following data:

- supplier
- number of copies.

9.4 ACTION AND PERCEPT DETAILED DESIGN

Actions and percepts have been identified during earlier stages. If there is processing of incoming data required to obtain percepts, this needs to be designed and specified. If a non-agent approach is being used to process the data (as is common in, say, image processing), then design can be done using a notation and methodology suitable for the approach being used. What is important is that the design of this part of the system is not forgotten, and, in particular, that wrong assumptions are not made as to what will/can be provided.

Like percepts, actions may be simple or complex. They may be simple system calls that invoke well-known and supported functionality, such as sending an e-mail or, alternatively, they may require customized design and implementation. Particularly in physical systems, actions often require significant work in order to obtain satisfactory behaviour. For example, in robocup, a move-to-point action required addressing issues such as how close to the point would be considered ‘at’ the point, what velocities and accelerations should be used, whether these should be parameters to the action and if so how they should be dealt with, as well as issues to do with the feedback loop between effectors and code. If actions are complex, then design documentation regarding them is an important part of the design of the system as a whole. Like percept processing, the notation and methodology used for designing the action components should be appropriate for the approach being used.

9.5 DATA

Data representation is an important part of any system design activity, and agent systems are no exception. However, as mentioned earlier, we do not focus on this aspect as existing techniques are suitable for this purpose. Nevertheless, data representations must be decided and specified, using some appropriate methodology.

Some agent development environments do offer some specialized support for data representation, such as the *beliefsets* available in JACK. If available, such specialized data representations can be useful as they typically provide some additional functionality. However, this is again an implementation platform issue. For arbitrary data structures, object-oriented class diagrams and access methods can be an appropriate representation.

What is important at the detailed design phase is to ensure that all significant data structures are well specified. All data required by plans should be identified and the location of that data should be specified.

9.6 DEVELOP AND REFINE DESCRIPTORS

The final design artifacts required are the individual descriptors that provide the detailed information for each item. The new descriptor type introduced during detailed design is the plan descriptor. Other descriptor types often obtain additional fields at this stage. For example, agents and capabilities need fields added to indicate which plans are included. Messages and percepts need to be extended with information about such things as coverage and overlap. In addition, many new instances will be added at this stage, in particular, many new internal messages representing sub-tasks, and many new data descriptors.

The descriptors provide the details necessary to move into implementation. Some of these details will depend on aspects of the implementation platform, while others are more generic. For example, if the context in which a plan type is to be used is split into two separate checks within the system being used (as is the case in JACK), then it is appropriate to specify these separately in the descriptor. Fields added to percepts and messages, specifying what information an event carries will depend on the structure of events within the system, and so on. The guiding principle is that these descriptors should (a) carry enough information for the reader to thoroughly understand the capability overview diagram, and (b) carry sufficient information to move directly to implementation.

The *plan descriptors* we use provide an *identifier*, the *triggering event type*, a *context* specification indicating when this plan should be used, the *plan steps* as well as a short natural language *description*, *messages* received and sent and a list of *data* used and produced. At this stage, data lists should always be specifically tied to particular data descriptors, rather than being generic descriptions.

The *coverage and overlap* information discussed earlier should be added to all message and percept descriptors. An event is *covered* if there is always at least one handling plan that is applicable; that is, for any situation, at least one of the matching plans will have a true context condition. There is *overlap* if it is possible to have multiple plan instances available to respond to this event. No overlap means that there is always at *most* one plan that is applicable in any situation. If there is overlap, then some notes

should be made regarding this, and the preferences between overlapping plans should be made clear, if there are any.

If the implementation system allows sub-typing of events, (as can, for example, be done using JACK's *relevance* field), then this should also be added as additional information in the message/percept descriptor.

Data descriptors should specify the fields and methods of any classes used for data storage within the system. If specialized data structures are provided for maintaining beliefs, these should also be specified. The data dictionary should also be updated and checked at this stage.

Electronic Bookstore: Case study

Following is an example of a plan descriptor for a plan to add a book to the stock order.

Plan: Add to order

Description: Order more stock of a book.

Trigger: Stock low

Context: Book not already ordered

Data used & produced: Reads Stock Orders and Monthly Order

Goal: Have books in stock

Failure: never fails

Failure recovery: not applicable

Procedure:

1. Determine how much stock to order (Get number required)
2. Determine which supplier to use (Decide supplier)
3. Add the order to the monthly order (Modify monthly order).

9.7 CHECKING FOR COMPLETENESS AND CONSISTENCY

As with *system specification* and *architectural design*, the *detailed design* also needs to be checked for completeness and consistency. As with previous phases, this is greatly assisted by the use of a tool that can automate a large part of this work. As with earlier phases, naming consistency is important, but we do not further discuss it here. At this stage, there is a proliferation of entities and many will have related names, so it is critical to know that only the exact same name refers to the same entity.

We structure the things that we check at this stage into four main areas:

1. Checking the completeness of each agent:
Does it implement the required functionality? Are all messages required for its specified participation in protocols generated and received?

2. Nothing is missing or redundant:

Ensure that all data is both used and produced (except for external data, where it is possible for it to be only used or produced). Ensure that all messages are both sent and received.

3. Checking the consistency between final design artifacts:

We need to ensure consistency of interfaces between levels. We require consistency between overview diagrams and descriptors, and we check consistency of plan details.

4. Following through important scenarios

As mentioned previously, these checks are a combination of horizontal checks, within the artifacts generated during this phase, and vertical checks with artifacts generated during previous phases.

9.7.1 AGENT COMPLETENESS

One of the most basic checks is to ensure that each agent is in fact covering the full range of functionality that was assigned to it in the architectural design. Often, each functionality is directly represented by a capability, in which case it is straightforward to ensure that no functionalities have been missed.

It is also a good idea to ensure that each message required by the protocol specifications in which the agent participates is in fact received or generated by the agent.

Plans implement the details of the functionality provided and therefore it is wise to carefully review the subsets of plans that implement particular processes. In particular, it is useful to review the set of plans that respond to a single trigger, to ensure that all situations are covered by some plan (or that it is acceptable if this is not the case). Overlap between plans can also be reviewed at this stage.

9.7.2 MISSING OR REDUNDANT ITEMS

As a system is developed, it is extremely common to end up with some redundant or missing information. As functionality is trimmed, or different ways are decided on for doing things, items that were previously needed are no longer relevant. As functionality is added, there is addition of items, but sometimes not all of them are fully integrated.

Missing information will of course show up in some way in the implemented system, possibly manifesting as some kind of an error. Redundancy, however, may never be noticed – resulting in additional work for no benefit during implementation, and code that is harder to maintain.

The specific items that we check in this area are messages, data, actions and percepts.

MESSAGES MUST BE BOTH SENT AND RECEIVED

All messages in the system must be ultimately sent from at least one plan. Similarly, all messages must be received by at least one plan. Usually this will be as a plan trigger,

although some messages may be received within a plan body. This check can be done by looking at plan descriptors and their incoming and outgoing messages. If an incoming message is not also a trigger, then the procedure should be checked to ensure there is a mechanism for receiving the message.

DATA MUST BE BOTH USED AND PRODUCED (USUALLY)

Usually, it is the case that if data is produced, we would want to see that it is also used. An exception may be if the data is being produced purely for external purposes, for example, a summary report of the number of storms in a year from a weather bureau, or a statistical report on sales from our electronic bookstore.

Certainly, data that is used must be produced or obtained somewhere. Data can come basically from three sources. It can be produced by some entity within the system; it can come from environmental information via percepts; or it can come from external data stores. If it is not being produced within the system, it is a good idea to check carefully the source.

ACTIONS AND PERCEPTS

It is of no use having actions defined (and perhaps implemented) that are never actually executed. Similarly, spending time inputting percepts, and perhaps collecting and processing data for these, is wasted effort if they are not used in some way.

During architectural design, it was established which agents were receiving which percepts and executing which actions. Now it is important to ensure that some plan within the relevant agent(s) actually executes each action. For each percept, it is important to ensure that within the relevant agent(s) it either triggers some plan (which is the most common case) or that it automatically updates some knowledge or belief store, without use of a plan, or that it is accessed from within a plan body. Checking off percepts that are used as plan triggers is easily automated via descriptors and/or overview diagrams. The remaining cases should be checked manually.

9.7.3 CONSISTENCY BETWEEN ARTIFACTS

As the design is developed, it is important to continue to monitor consistency between the interfaces at the more abstract and the more detailed levels, as well as consistency between different kinds of representations.

CONSISTENCY BETWEEN OVERVIEW DIAGRAMS

The interfaces between the various overview diagrams can be simply and effectively checked for consistency. All incoming and outgoing links from an agent in the system overview diagram must translate to the same incoming and outgoing links within the

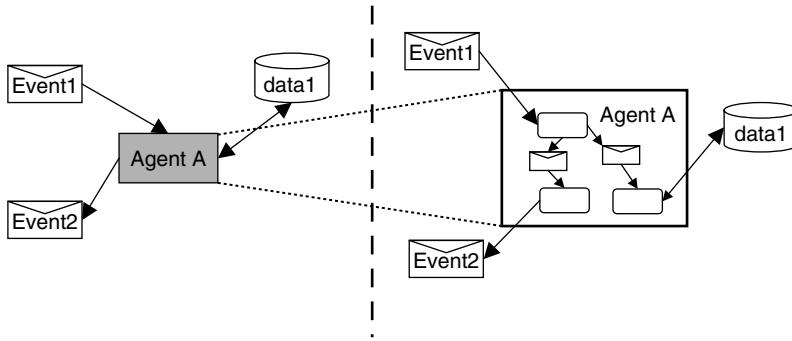


Figure 9.4 Interfaces should match for every entity at different diagram levels

agent overview diagram. Similarly, the incoming and outgoing links from a capability within an agent overview diagram must be identical to the incoming and outgoing links in the capability overview diagram for that capability. The same principle applies to nested capabilities. Figure 9.4 illustrates this principle: the events and data that form the agents interface in the system overview diagram (left) also appear in the agent overview diagram (right).

This check sounds very trivial, but it is surprising how often an interface item is inadvertently added or dropped.

 **TIP:** The authors have found that large numbers of minor errors and omissions that would most likely percolate into code are easily found by mechanical and routine consistency checking, much of which can be automated. We have also found that design documents with widespread inconsistencies are extremely difficult to understand if one has not been involved in the design process.

CONSISTENCY BETWEEN OVERVIEW DIAGRAMS AND DESCRIPTORS

Consistency between all overview diagrams and the relevant descriptors should be performed, in a manner similar to that described in Section 7.6.4. Capability descriptors should be checked against the agent overview diagram, or in the case of nested capabilities, against the relevant capability overview diagram. If a capability is used in more than one place, then it should be checked against all relevant overview diagrams.

Plan descriptors should be checked against the capability overview diagram (or in some cases, the agent overview) to ensure that incoming and outgoing messages and data uses are consistent. With regard to plan descriptors, it is also useful to check some aspects against the procedure description. For example, it should be evident within the procedure description where each outgoing message is posted. Similarly, for any incoming message that is not the trigger, it should be evident where it is accessed. A similar check can be made with respect to actions and percepts if relevant.

Descriptors for messages, data stores, percepts and actions should also be checked for consistency with overview diagrams.

CONSISTENCY OF SCENARIOS, PROCESSES AND PLAN SETS

Scenarios were developed at an early stage to capture important aspects of the system functionality. Before detailed design can be considered complete, a check should be made to ensure that these scenarios are captured by processes and plan sets within each of the relevant agents. It is not uncommon that in the process of detailed design, scenarios do change somewhat. Depending on the project, it can be important to update the scenarios, as they are important in helping to understand a design. If they are not updated, they should at least be marked as no longer current when inconsistencies are detected, and it is determined that it is the scenario that should be changed.

PROCESSES CONSISTENT WITH PROTOCOLS AND OVERVIEW DIAGRAMS

Processes are a detailed description of the dynamics of various activities within a particular agent. They should be checked especially against the protocol related to the activity, particularly to ensure that messages in and out are consistent with the message exchanges indicated in the relevant protocol. Similarly, triggers for the activity should be found (as percepts or messages) on the agent overview diagram, unless they are confirmed to originate within the agent.

These checks are not readily automatable, but structured manual checking is quite effective.

9.7.4 IMPORTANT SCENARIOS

It can be useful to do a ‘design walk through’ of key scenarios together with a colleague. Flaws can often be seen at this stage and it is preferable to rethink and remedy them within the design.